

One To One Relationship

We need **1:1 relationships** when:

- We want to **separate optional/sensitive data**.
- We want to **optimize performance**.
- The **real-world model naturally enforces a one-to-one rule**.

when we should use 1:1 relationship and how to identify give me example in term of sql and nosql mongodb database

When should we use 1:1 relationships?

You should use **1:1** when:

1. **Data is optional** → Not every entity needs those extra details.
 - Example: A `user` may or may not have a `driving_license`.
2. **Data is sensitive / access-controlled** → Separate storage with stricter permissions.
 - Example: Storing **passwords or medical records** apart from general info.
3. **Performance reasons** → To avoid a "fat table" with rarely used fields.
 - Example: `employees` table queried often, but medical details are rarely needed.
4. **Logical domain separation** → Different life cycles of data.
 - Example: `users` table is permanent, but `profiles` may be deleted/updated separately.
5. **Real-world concept enforces 1:1** → Naturally one-to-one in business rules.

- Example: A **passport belongs to exactly one person**.

● How to Identify a 1:1 Relationship in SQL (Relational DBs)

Ask yourself:

1. **Does each row in Table A map to exactly one row in Table B?**
 - Example: One `user` → one `profile`.
2. **Will Table B's row exist only if Table A's row exists?**
 - Example: A `profile` cannot exist without a `user`.
3. **Do I want to split optional/sensitive columns into a separate table for normalization?**
 - Example: Store `medical_details` in a separate table instead of inside `employees`.

👉 If yes, you model it with **two tables + a `UNIQUE` foreign key constraint**.

● How to Identify a 1:1 Relationship in MongoDB (NoSQL)

Ask yourself:

1. **Does each document in Collection A link to exactly one document in Collection B?**
 - Example: One `user` document → one `profile` document.
2. **Can Collection B's document exist without Collection A's document?**
 - Example: A `profile` should not exist without its `user`.

3. Should I embed the data (if always present & small) or reference it (if optional, sensitive, or large)?

- **Embed** → if profile is always needed (tight coupling).
- **Reference** → if profile is optional, big, or secured separately.

👉 If yes, you enforce it with either:

- **Embedded schema** inside the parent document.
- Or a **referenced schema** with `{ unique: true }` on the foreign key.

● SQL Example (Relational DB)

Users & Profiles (separating optional profile info)

```
-- Main users table
CREATE TABLE users (
  user_id INT PRIMARY KEY AUTO_INCREMENT,
  email VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL
);

-- Profile table (1:1 with users)
CREATE TABLE profiles (
  profile_id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT UNIQUE, -- UNIQUE enforces 1:1
  bio TEXT,
  dob DATE,
  avatar_url VARCHAR(255),
  FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

- Each `user` → at most one `profile`.

- `user_id` in `profiles` is marked **UNIQUE**, so you can't attach multiple profiles to the same user.

● MongoDB Example (NoSQL)

MongoDB gives **two approaches**: **embedding** vs **referencing**.

1 Embedding (if profile info is always present and small)

```
{
  "_id": ObjectId("..."),
  "email": "john@example.com",
  "password_hash": "...",
  "profile": {
    "bio": "Software developer",
    "dob": "1998-05-21",
    "avatar_url": "http://example.com/john.png"
  }
}
```

- This keeps everything in **one document**.
- Use this if **profile is always there** and small.

2 Referencing (true 1:1, separate collection)

```
// users collection
{
  "_id": ObjectId("user123"),
  "email": "john@example.com",
  "password_hash": "..."
}

// profiles collection
{
```

```
"_id": ObjectId("profile123"),
"user_id": ObjectId("user123"), // Reference to user
"bio": "Software developer",
"dob": "1998-05-21",
"avatar_url": "http://example.com/john.png"
}
```

- Here `user_id` is **unique** in `profiles`.
- Makes sense if **profile is optional** or **access-controlled**.

⚡ Rule of Thumb:

- **SQL** → enforce with `UNIQUE` + `FOREIGN KEY`.
- **MongoDB** →
 - **Embed** if data is always together, small, and low update frequency.
 - **Reference** if optional, sensitive, or large.

In a 1:1 relationship, which collection should store the reference?

◆ General Rule for 1:1 in Mongoose

- The reference should usually go in the collection that represents the **dependent / optional** side.
- The **main (parent)** entity stays clean.

Example: User ↔ Profile

Option 1: Reference in `Profile` (Recommended ✅)

```
// profiles collection
{
  _id: ObjectId("profile123"),
  user: ObjectId("user123"), // reference to User
  bio: "Software Dev",
  dob: "1998-05-21"
}
```

👉 Why this is better:

- **Every user must exist first**, profile is optional.
- If the profile is deleted, user still exists.
- Enforce 1:1 with `unique: true` on `user` field.

Mongoose schema:

```
const profileSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: "User",
    unique: true },
  bio: String,
  dob: Date
});
```

Option 2: Reference in `User` (Alternative, when profile is always required)

```
// users collection
{
  _id: ObjectId("user123"),
  email: "john@example.com",
  profile: ObjectId("profile123") // reference to Profile
}
```

```
}
```

👉 Why this is less common:

- If profile is optional, you'll end up with `null` values.
- Makes queries heavier because now every user carries a profile reference.
- Good only if **every user must have exactly one profile** and you often fetch both together.

Mongoose schema:

```
const userSchema = new mongoose.Schema({  
  email: { type: String, required: true, unique: true },  
  profile: { type: mongoose.Schema.Types.ObjectId, ref: "Profile" } // 1:1  
});
```

◆ How to Identify Which Way to Go?

Ask these 3 questions:

1. Which entity is primary?

- `User` is primary → `Profile` depends on it.
- So reference usually goes in **Profile**.

2. Is the secondary entity optional?

- If optional → reference goes in the **optional side** (`Profile`).
- If always required → you can safely put it in `User`.

1. Who owns the lifecycle?

- If deleting a user should delete the profile → reference in `Profile` is cleaner.

- If profile owns user (rare case, like `passport → person`), reference could be in `User`.

Example and use case of 1:1 relationship

10 real-life 1:1 relationship



- **User → Profile**
`users` collection holds `profileId`
- **User → Address**
`users` collection holds `addressId`
- **Order → Payment**
`orders` collection holds `paymentId`
- **Employee → BankDetails**
`employees` collection holds `bankDetailsId`
- **Student → ReportCard**
`students` collection holds `reportCardId`
- **Product → Inventory**
`products` collection holds `inventoryId`
- **Vehicle → Registration**
`vehicles` collection holds `registrationId`
- **Device → Configuration**
`devices` collection holds `configurationId`
- **Account → KYC**
`accounts` collection holds `kycId`